

03 | Bottlerocket：为云原生而生的强大操作系统

潘野 · 云原生基础架构实战课



你好，我是潘野。

在做云原生训练营助教的时候，很多同学对“不可变基础设施”这一个概念不太理解，如果你在网络上搜索“不可变基础设施”，往往会到得这样的说法：

不可变基础设施是指一种基于代码和配置的基础设施管理方法，其中所有的更改和配置都是通过重新构建和替换来实现的，而不是直接修改现有的设施。这种方法的目标是确保系统的稳定性和一致性，减少人为错误和配置漂移，并提高系统的可靠性和可管理性。

很多同学读了之后一头雾水，产生了后面这些疑问。

我用 puppet 或者 ansible 不也是代码方式来管理基础设施，这样不是也能保证一致性吗？

如果更改配置都是通过替换实例的方式，这样做的成本是不是太高了？

没看出“不可变”的好处在哪里？

今天这节课，我就带你重新理解一下“不可变基础设施”，看看它究竟带来了什么样的管理优势。最后，我还会带你一起体验下，不可变基础设施中的新星 bottlerocket 如何使用。

运维之痛：给操作系统打补丁

通过前面的学习，我们知道了在虚拟机时代，的确可以通过 Puppet、Saltstack、Ansible 这类配置管理工具来实现 IaC。即便到今天，这些工具也仍然在运维管理工作中非常流行。

以往比较常见的运维工作就是给虚拟机打安全补丁，或者是升级主机内核，这时我们的做法基本都是更改 Puppet 的配置文件，配置推送到目标机器，并且联系业务团队安排时间做主机重启。用这种方式管理的机器，我们叫做**可变基础架构**。

如果公司里机器数量是几百、几千台，通过自己开发的运维平台，投入三四个人力，花上半个月基本也可以搞定。但是如果你公司里的机器数量在十万台以上，你会发现必须安排几个运维同学，常年给操作系统打补丁。

在虚拟机时代，一个机器上一般只跑一个应用，为了提高资源利用率，最多也就跑三四个应用，那么通知用户、给机器打补丁，这些事可能还属于可控范围内。然而现在的容器环境中，一个主机上动辄几十到上百个容器，并且掺杂了很多不同的应用。这时候工作变得不太可控，三四个人可能半年也做不完一轮维护。

近些年安全管理在各个公司里都逐渐上升到公司战略层面，安全管理中最常见的一个事情就是给操作系统打安全补丁；而且自从 Docker 出现之后，Linux 操作系统的内核版本演进变得快了很多，新的内核中包含了很多为容器设计的特性，比如最近非常热门的 eBPF，它要求 Kernel 的版本不能低于 4.1，而真正比较完善的是 4.18 这个版本，所以 Kernel 版本的跟进也成为了基础架构管理中的一个强需求。

那么在云原生时代，如果我们还是依靠原来的管理方式，通过 Puppet 等配置管理工具滚动地升级内核版本、重启主机，这不仅非常耗费人力，而且还可能会造成一些应用的中断，并不是一个很优雅的管理方式。

既然云原生时代，容器这种部署稳定可靠且易管理的方式已经深入人心，我们能不能像管理容器一样来管理虚拟机，来简化运维操作呢？

什么是不可变基础设施？

想要像管理容器一样管理虚拟机，我们需要先分析一下容器有哪些特点。

我们第一反应大概会包括这几个特点：能快速启动、轻量级、有版本控制等等。

所以对应到虚拟机上，我们就会有这样的期望。

虚拟机启动要越快越好，所以镜像要最小化。

虚拟机镜像是有版本控制的，当虚拟机需要打补丁的时候，我们需要做的是，打包编译新的镜像，然后用新镜像启动机器做直接替换。

利用 Kubernetes 自身的能力，保证 pod 平滑地漂移到新的节点上即可。一切的管理都由 Kubernetes 来负责，不再需要人工介入管理。

我们期望的这种管理方式其实就是不可变基础设施！

有了感性认知以后，我们就可以重新拆解不可变基础设施的定义了，这里有三个关键点需要掌握。

关键点一，一种基于代码和配置的基础设施管理方法。

我们打包一个容器镜像的过程是，写 Dockerfile，编译镜像，推送进容器仓库。一般这个过程都是由公司内的 CI/CD 工具来完成。其实虚拟机的镜像管理也是类似的做法，利用 Packer 这样的打包工具，配合 CI 工具来生成虚拟机镜像，然后推送到虚拟机镜像管理的仓库。

关键点二，所有的更改和配置都是通过重新构建和替换来实现的。

当 v1 版本的虚拟机需要更新到 v2 版本的虚拟机，我们只要先启动 v2 版本的虚拟机，然后将 v1 版本虚拟机中的 workload 转移到新机器即可。我估计，熟悉 Kubernetes 的同学立刻就可以想到有哪些方法可以转移 workload。

关键点三，确保系统的稳定性和一致性，减少人为错误和配置漂移。

运维的同学应该深有体会，面对几万台机器的管理，人为推错了配置这种情况并不少。另外，不健壮的配置脚本也可能引起环境配置不一致。而实施不可变基础设施的最大作用，就是避免这类情况出现。

好，前面我们花了不少篇幅讲解不可变基础设施的概念，为了让你加深对它的认识，我们来看看历史上曾经出现的不可变基础设施的技术。

CoreOS/Atomic – 出师未捷

我们都知道 Kubernetes 的核心组件是 etcd。最早发布 etcd 的这家公司叫 CoreOS，公司的名字就是自家的一个操作系统 CoreOS。CoreOS 为容器专门设计操作系统的鼻祖，可能你没用过甚至没听说过它，所以我简单做个介绍。

CoreOS 是在 2013 年出现的，CoreOS 比 Kubernetes 还早出来一年，CoreOS 就是为“不可变基础设施”量身打造的操作系统，它与传统的 CentOS/Ubuntu 这类系统不同，具有以下特点。

CoreOS 利用主动和被动双分区方案来更新 OS，使用分区作为一个单元，而不是一个包一个包的更新。这使得每次更新变得快速、可靠，而且很容易回滚。

CoreOS 内置了服务发现工具 etcd，容器管理工具 rkt。

CoreOS 专门为云环境设定，提供了 AWS、GCP、Azure 等各大云厂商镜像。

作为 Linux 操作系统的扛把子 Redhat，看到 CoreOS 的设计方式之后大为震惊，基于 RHEL 的系统也推出了专为容器设计的操作系统 Atomic，eBay 的容器平台使用的就是 Atomic。

2018 年的时候，云原生热潮席卷全球，Red Hat 干脆直接收购了 CoreOS，并且将 CoreOS 与 Red Hat 原有的 Atomic 操作系统项目合并，一并改名叫 Fedora CoreOS。

这两个操作系统的特点，非常符合前面所说的不可变基础设施的最佳实践，但是两个项目最终合并成为了商业产品。那么在容器时代，还有没有其他可以不花钱的替代产品了？

有，这就是 Amazon 专门推出的、为容器服务的操作系统 Bottlerocket！

Bottlerocket – 旗开得胜

Bottlerocket 是 Amazon 在 2020 年 3 月时发布的一套完全开源的操作系统，由现有开源组件（例如 Linux 内核）、约 50 个软件包以及专为 Bottlerocket 编写的新组件（主要以 Rust 及 Go 语言编写而成）共同组成。你可以在 AWS 网站上找到它们的 [介绍](#)。

你可能仍然有顾虑，前面提到 CoreOS 被收购了，Atomic 项目就终止了，那么 Bottlerocket 有没有这样的风险呢？

从现在的情况来看，Bottlerocket 中 Amazon 自己编写的组件使用 Apache 2.0 许可或 MIT 许可，Linux Kernel 等其他开源组件遵守原始开源许可，所以风险不大。同时 Bottlerocket 项目源自长期以来在 Amazon 大规模生产运行服务当中总结出的经验教训，结合了过去多年以来 Amazon 对于容器技术的提炼与沉淀，可以说 Bottlerocket 是现在容器操作系统的绝佳选择。

Bottlerocket 和传统操作系统相比，引入了一些新的概念，这里我着重讲一下 Bottlerocket 的三大特点。

第一，Bottlerocket 没有软件包管理器，一切应用程序用容器运行。

很多同学不太理解什么是“没有包管理器”。传统的操作系统，比如 Ubuntu 或者 CentOS，它们的包管理器是 RPM 和 DEB，基于 RHEL 系统的 Atomic 就是用的 RPM 包管理的模式。然而，在 Bottlerocket 没有任何的包管理工具，Bottlerocket 系统里的组件是直接以二进制形式打包进镜像的。这样做可以减少很多无用组件，达到操作系统最小化的目的。

第二，Bottlerocket 是一个 API 驱动操作系统。

“API 驱动”，这听起来也比较难理解。简单来说就是当操作系统启动之后，系统没有 ssh，也没有传统 Linux 上必备的 shell。如果我们想调试系统，需要启动一个带有 shell 的容器来调试、手动更新和更改主机上的配置。那么启动这个 shell 的容器，就是调用 Bottlerocket 的 API 来启动这个 shell，在后面的章节我会演示如何使用它的 API，这里你先有个印象就好。

第三，Bottlerocket 根文件系统是只读，不可修改，这是实现“不可变基础设施”的一个重要特性。而非根分区系统也就是用户存数据的分区，可以读写，采用 SELinux 策略来保护非根文件系统中的文件。

快速体验

说了这么多 Bottlerocket 的特点，我们怎么快速上手呢？

我猜有同学已经通过搜索点开了 Bottlerocket 的 [🔗 GitHub 链接](#)，但是可能会发现 GitHub 里一些指引看不明白，比如说，当我们接触一个新的操作系统，第一步肯定是在虚拟机里尝试安装，但是你在 Bottlerocket 的 GitHub 上找不到“如何安装 Bottlerocket”这样的文档。

这是因为 Bottlerocket 是为容器而生，所以整个的设计是围绕 Kubernetes 的，所以它的使用离不开 kubernetes 集群。如果想体验 Bottlerocket，最快捷的办法是在 AWS 上启动一个 EKS 集群，然后选择 Bottlerocket 作为节点的操作系统。

当然，如果你要想把它用在物理机器或者在本地环境里跑起来，Bottlerocket 也可以提供了相应的文档指引，你可以自行查阅 Github 上的 [🔗 这个链接](#)了解更多内容。在 Bottlerocket 的 Github 上，Bottlerocket 不但为 Kubernetes 1.23 之后的每个主版本都提供了镜像，同时还提供了 AWS AMI 镜像和 VMware 的镜像。

调试实操环节

最后，我们来简单体验一下怎么进入 Bottlerocket 的调试模式，并且尝试一些重启机器之类的操作，找找手感。

首先，我们在 EKS 集群里添加一个节点组，其中 AMI 的类型是 Bottlerocket_x86_64，你可以参考后面的截图来配置。

[EKS](#) > [集群](#) > [hilarious-wardrobe-1702224307](#) > 添加节点组

第 1 步
[配置节点组](#)

第 2 步
设置计算和扩展配置

第 3 步
指定联网

第 4 步
查看和创建

设置计算和扩展配置

节点组计算配置

创建节点组后，无法更改这些属性。

AMI 类型 信息

为节点选择 EKS 优化的 Amazon Machine Image。

Bottlerocket (BOTTLEROCKET_x86_64)	▲
Amazon Linux 2 (AL2_x86_64)	
已启用 Amazon Linux 2 GPU (AL2_x86_64_GPU)	
Amazon Linux 2 Arm (AL2_ARM_64)	
Bottlerocket (BOTTLEROCKET_x86_64)	✓
Bottlerocket Arm (BOTTLEROCKET_ARM_64)	
Bottlerocket NVIDIA (BOTTLEROCKET_x86_64_NVIDIA)	
Bottlerocket Arm NVIDIA (BOTTLEROCKET_ARM_64_NVIDIA)	
Windows Server 2019 Core (WINDOWS_CORE_2019_x86_64)	
Windows Server 2022 Core (WINDOWS_CORE_2022_x86_64)	
Windows Server 2019 Full (WINDOWS_FULL_2019_x86_64)	
Windows Server 2022 Full (WINDOWS_FULL_2022_x86_64)	



添加完成之后等待片刻，我们就可以在集群里看到一个新的节点产生。之后用 `-owide` 参数就可以看到这个节点的操作系统是 Bottlerocket。

复制代码

```
1 ~>kubectl get node ip-192-168-81-115.ap-east-1.compute.internal -owide
2 NAME                                     STATUS    ROLES    AGE      VERSION
3 ip-192-168-81-115.ap-east-1.compute.internal Ready     <none>    3m47s    v1.27.7-
```

接下来，让我们在 AWS 的 console 上找到这个 EC2 节点，使用 session manager 这个工具连接这个节点。然后你会看到 Bottlerocket 的欢迎提示，同时给你提示了三个命令。

第一个命令是 `apiclient`，这是用来与 Bottlerocket API 交互的工具。

第二个命令是 `enter-admin-container`，它的作用是启动一个具有 admin 权限的容器，我们接下来的操作都会在 admin 容器中进行。

第三个命令是 `disable-admin-container`，该命令用来关闭具备 admin 权限的容器。



Welcome to Bottlerocket's control container!

This container gives you access to the Bottlerocket API, which in turn lets you inspect and configure the system. You'll probably want to use the ``apiclient`` tool for that; for example, to inspect the system:

```
apiclient -u /settings
```

You can run ``apiclient --help`` for usage details, and check the main Bottlerocket documentation for descriptions of all settings and examples of changing them.

If you need to debug the system further, you can use the admin container. The admin container has more debugging tools installed and allows you to get root access to the host. The easiest way to get started is like this, which enables and enters the admin container using `apiclient`:

```
enter-admin-container
```

You can also access the admin container through SSH if you have network access. Just enable the container like this, then SSH to the host:

```
enable-admin-container
```

You can disable the admin container like this:

```
disable-admin-container
```

```
[ssm-user@control]$
```

敲下 `enter-admin-container` 命令之后，我们即可进入 admin 容器。然后敲下 `apiclient reboot` 命令，此时你会发现提示机器重启了，集群中这个节点变成了 NotReady 的状态。等机器启动完成之后，这个节点会重新变成 Ready 状态。

```
[ssm-user@control]$ enter-admin-container
Confirming admin container is enabled...
Waiting for admin container to start...
Entering admin container
    Welcome to Bottlerocket's admin container!


    This container provides access to the Bottlerocket host
    filesystems (see /.bottlerocket/rootfs) and contains common
    tools for inspection and troubleshooting. It is based on
    Amazon Linux 2, and most things are in the same places you
    would find them on an AL2 host.

    To permit more intrusive troubleshooting, including actions that mutate the
    running state of the Bottlerocket host, we provide a tool called "sheltie"
    (`sudo sheltie`). When run, this tool drops you into a root shell in the
    Bottlerocket host's root filesystem.
    [root@admin]# apiclient reboot
    16:40:24 [INFO] Rebooting, goodbye...
    [root@admin]# [ssm-user@control]$
```

 极客时间

```
~ >>> kubectl get node
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-17-60.ap-east-1.compute.internal	Ready	<none>	22m	v1.27.7-eks-e71965b
ip-192-168-66-171.ap-east-1.compute.internal	Ready	<none>	22m	v1.27.7-eks-e71965b
ip-192-168-81-115.ap-east-1.compute.internal	NotReady	<none>	9m43s	v1.27.7-eks-1670f88
ip-192-168-96-252.ap-east-1.compute.internal	Ready	<none>	9m44s	v1.27.7-eks-1670f88

 极客时间

短短几分钟我们就体验了 Bottlerocket 的启动，API 交互的方法。如果你有兴趣，可以进入 admin 容器，运行 `apiclient -h` 来仔细研究一下 `apiclient` 这个命令能做什么事情。下面这张图展示了该命令支持的参数，供你参考。

```
[root@admin]# apiclient -h
Usage: apiclient [SUBCOMMAND] [OPTION]...

Global options:
  -s, --socket-path PATH      Override the server socket path. Default: /run/api.sock
  --log-level                 Desired amount of output; trace|debug|info|warn|error
  -v, --verbose               Sets log level to 'debug'. This prints extra info,
                              like HTTP status code to stderr in 'raw' mode.

Subcommands:
  raw                         Makes an HTTP request and prints the response on stdout.
                              'raw' is the default subcommand and may be omitted.
  apply                       Applies settings from TOML/JSON files at given URIs,
                              or from stdin.
  get                         Retrieve and print settings.
  set                         Changes settings and applies them to the system.
  update check                Prints information about available updates.
  update apply                Applies available updates.
  update cancel               Deactivates an applied update.
  reboot                      Reboots the host.
  exec                       Execute a command in a host container.
  report cis                  Retrieve a Bottlerocket CIS benchmark compliance report.
  report cis-k8s              Retrieve a Kubernetes CIS benchmark compliance report.

raw options:
  -u, --uri URI               Required; URI to request from the server, e.g. /tx
  -m, -X, --method METHOD     HTTP method to use in request. Default: GET
  -d, --data DATA            Data to include in the request body. Default: empty

apply options:
```

 极客时间

总结

不可变基础设施是云原生时代一种构建和管理基础设施的方法，这种方法提供了可靠性、可伸缩性和可重复性。而 Bottlerocket 是一个轻量级，高度可靠性和安全性的 Linux 发行版，是对不可变基础设施的最佳实践。

今天，我们学习了不可变基础设施的概念，其中不可变基础设施概念的三个关键点你需要重点掌握。之后，我们还通过实操体会到 Bottlerocket 的一些用法，还有其他的一些特点，同学们可以课后自行查阅 Bottlerocket 的 [官方文档](#)。建议你课后对照课程讲解，动手多多练习，加深记忆。

后面的课程中，我们启动的集群会默认使用 Bottlerocket 作为 kubernetes nodes，相信随着课程学习的深入，你也会对 Bottlerocket 更加熟悉。

思考题

这一讲，我提到容器环境中重启主机这有可能会造成一些应用的中断。你知道什么样的情况会造成应用中断么？我们又该如何预防？

欢迎你和我在评论区一起讨论。如果这一讲对你有帮助，别忘了分享给身边更多同事、朋友。

AI智能总结

1. 不可变基础设施是一种基于代码和配置的基础设施管理方法，旨在确保系统的稳定性和一致性，减少人为错误和配置漂移，提高系统的可靠性和可管理性。
2. 传统的基础设施管理方式可能会造成人力成本高、应用中断等问题，因此需要转向不可变基础设施管理方式，以简化运维操作。
3. 不可变基础设施的关键特点包括基于代码和配置的管理方法、通过重新构建和替换来实现更改和配置、确保系统的稳定性和一致性，减少人为错误和配置漂移。
4. Bottlerocket是Amazon推出的为容器服务的操作系统，是一个可以不花钱的替代产品，适用于云原生时代的不可变基础设施管理。
5. Bottlerocket的三大特点包括没有软件包管理器，是一个API驱动操作系统，根文件系统是只读，不可修改，这是实现“不可变基础设施”的一个重要特性。
6. Bottlerocket是一个轻量级，高度可靠性和安全性的Linux发行版，是对不可变基础设施的最佳实践。
7. Bottlerocket的使用离不开Kubernetes集群，可以在AWS上启动一个EKS集群，然后选择Bottlerocket作为节点的操作系统。
8. Bottlerocket的API交互工具和调试模式的使用方法，以及如何预防容器环境中重启主机可能造成的应用中断等相关内容。
9. 不可变基础设施的概念和Bottlerocket的特点是云原生时代基础设施管理的重要内容，对于提高系统的可靠性和可管理性具有重要意义。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (5)

最新 精选



暴躁的蜗牛

2024-04-20 来自广东

bottlerocket 是可以替代 容器的 Alpine Linux吗 感觉虽然都是操作系统 是否某些应用会指定操作系统的版本 这样对 系统bottlerocket 有影响吗

作者回复: Hi 蜗牛同学

bottlerocket目标还是做host os, 不是做容器base os。



👍 2



alex run

2024-04-13 来自上海

老师, 能谈一下私有云落地bottlerocket的做法吗? 然后我想问一下, 将传统操作系统替换成bottlerocket是有必要做的一件事情吗? 回报会高于投入吗?

作者回复: Hi Alex

GitHub上提供了私有云落地bottlerocket的一些指导 (<https://github.com/bottlerocket-os/bottlerocket/blob/develop/QUICKSTART-VMWARE.md>), bottlerocket的特点是面向kubernetes, 所以你需要先有一个kubernetes控制平面, 然后用bottlerocket的API向内注入一些配置, 之后完成整个provision的过程。

投入产出比要看几个因素, 私有云内集群规模有多大, 容器数量有多少, 团队成员的数量是否能在前期支撑转换过程等等。综合考量这些因素



👍 2



三万棵树

2024-03-29 来自北京

Bottlerocket 这个在国内的落地情况 想咨询您一下

作者回复: 同学你好, Bottlerocket在AWS的容器方向已经广泛使用, 我们也在研究将Bottlerocket带到私有云的环境中。从性能与管理上, 我们对Bottlerocket的表现感到满意。

共 2 条评论 >



二十三

2024-04-19 来自上海

国内公有云貌似都还没有支持





cfanbo

2024-03-29 来自浙江

Bottlerocket 这个在国内用的多吗？

